

Projet de Fin de Module : "Build & Ship" - Architecture Cloud & Vibe Programming

Pondération : 40% de la note finale du module

Format : Binôme (2 étudiants)

Technique: VIBE Coding

Stack technologique imposé :

- ☐ Frontend libre (Next.js, React, Vue...)
- ☐ **Supabase** (Base de données/BaaS)
- ☐ **Vercel** (Hébergement CI/CD).

Date remise : 10/05/2026



La Philosophie du Projet : "Vibe Coding"

Dans ce projet, fini la configuration fastidieuse de serveurs physiques. Vous allez coder avec les standards de l'industrie : **prototyper vite, utiliser les meilleurs outils cloud, et déployer en un clic.**

Votre objectif est de concevoir, développer et mettre en production un **Extranet métier** fonctionnel, tout en justifiant l'architecture Cloud (Serverless) qui se cache derrière.



La Règle d'Or : "Même Squelette, Habillage Différent"

Pour éviter le plagiat, **chaque groupe travaillera sur un thème métier différent.** Cependant, l'exigence technique est strictement la même pour tout le monde.

Quelle que soit votre application, vous devrez obligatoirement modéliser cette architecture dans Supabase :

- **Table A (Utilisateurs)** : Gérée via Supabase Auth (Les clients, locataires, patients...).
- **Table B (Ressources)** : Les éléments consultables (Les véhicules, maisons, médecins...).
- **Table C (Interactions)** : La table de jointure qui relie A et B avec une date/statut (Les réservations, rendez-vous, contrats...).
- **Storage (Fichiers)** : Un document uploadé lié à l'interaction (Un PDF, un scan de carte d'identité, une image...).



La Roulette des Thèmes

Inscrivez votre binôme sur le thème de votre choix (premier arrivé, premier servi) :

1. **Immobilier ("Dar-Connect")** : (A) Locataires, (B) Maisons, (C) Visites. Fichier = Scan de la carte d'identité.
2. **Location Auto ("Auto-Loc")** : (A) Clients, (B) Voitures, (C) Réservations. Fichier = Photo du permis.
3. **Support Informatique ("IT-Fix")** : (A) Employés, (B) Techniciens, (C) Tickets de panne. Fichier = Capture d'écran du bug.
4. **Éducation ("Dourous-Net")** : (A) Élèves, (B) Professeurs, (C) Séances. Fichier = PDF du devoir.
5. **Logistique B2B ("Fret-Dz")** : (A) Commerçants, (B) Camionneurs, (C) Expéditions. Fichier = Bon de livraison signé.
6. **Recrutement ("Talent-DZ")** : (A) Candidats, (B) Offres d'emploi, (C) Candidatures. Fichier = CV en PDF.
7. **Copropriété ("Batima-Gest")** : (A) Résidents, (B) Parties communes, (C) Signalements pannes. Fichier = Photo du problème (ex: ascenseur).
8. **Juridique ("Avocat-Link")** : (A) Clients, (B) Avocats, (C) Consultations. Fichier = Dossier de preuve (PDF).
9. **Événementiel ("Saha-Event")** : (A) Clients, (B) Salles des fêtes, (C) Réservations. Fichier = Reçu de paiement CCP.
10. **Clinique Vétérinaire ("Veto-Care")** : (A) Maîtres, (B) Vétérinaires, (C) Rendez-vous. Fichier = Carnet de santé de l'animal.



Les 4 Missions du Binôme

Mission 1 : La Data & La Sécurité (Supabase) -> *Lien avec le SGBD*

Vous utiliserez Supabase (PostgreSQL Cloud).

1. **Modélisation SQL** : Créez vos 3 tables (A, B et C) avec les clés étrangères appropriées.
2. **Auth (Extranet)** : Implémentez l'authentification. L'application doit obliger l'utilisateur de la Table A à se connecter pour utiliser le service.

3. **Row Level Security (RLS) : Critère éliminatoire.** Configurez la sécurité dans Supabase pour qu'un utilisateur ne puisse voir *que* ses propres interactions (Table C). (Ex: Un client de location auto ne doit pas voir les réservations des autres clients).
4. **Données non-structurées :** Utilisez *Supabase Storage* pour permettre l'upload du fichier requis par votre thème.

Mission 2 : Le Frontend & L'Expérience Utilisateur

1. Développez l'interface utilisateur. L'utilisation d'IA (Cursor, Copilot, v0) est autorisée pour accélérer la conception visuelle (Vibe Coding !).
2. L'application doit permettre un flux complet : Inscription/Connexion -> Consulter les ressources (Table B) -> Créer une interaction (Table C) avec upload de fichier -> Voir son tableau de bord personnel.

Mission 3 : Le Déploiement CI/CD (Vercel) -> Lien avec DevOps

1. Poussez votre code sur un dépôt **GitHub** partagé par le binôme.
2. Connectez ce dépôt à **Vercel**.
3. À chaque `git push`, Vercel doit re-builder et déployer votre site. L'application doit être accessible via un lien public fonctionnel (ex: `votre-projet.vercel.app`).

Mission 4 : Le README "Architecte" (Markdown)

Dans le fichier `README.md` de votre projet GitHub, intégrez ces deux éléments :

1. **Le Mapping de votre Thème :** Décrivez brièvement votre thème et indiquez clairement à quoi correspondent vos Tables A, B, C et votre Fichier (pour faciliter la correction).
 2. **L'Analyse d'Architecture (500 mots max) :** Faites le lien avec le cours en répondant à ces 3 questions :
 - Pourquoi l'utilisation de Vercel + Supabase est financièrement plus logique pour lancer ce projet qu'un serveur classique ? (Utilisez les termes **OPEX** et **CAPEX**).
 - Comment Vercel gère-t-il la scalabilité par rapport à un Data Center physique local (Climatisation, serveurs rack...) ?
 - Dans votre application, qu'est-ce qui représente la donnée **Structurée** et la donnée **Non-structurée** ?
 - 3.
-



Livrables Attendus & Rendu

Envoyez par email (ou via la plateforme de l'école) un document PDF d'une page contenant :

1. **Les Noms & Prénoms** du binôme et le **Thème choisi**.
2. **L'URL de production** de l'application déployée.
3. **Le lien vers le dépôt GitHub public**.
4. **Des identifiants de test** (un email et mot de passe d'un faux utilisateur déjà créé pour que l'enseignant puisse tester directement).



Grille d'Évaluation (sur 20 points)



Critères d'évaluation	Points	Description des attentes
1. Fonctionnalité (Le "Ship")	/5	L'application est en ligne (Vercel). Le flux (Auth -> Création Table C -> Upload) fonctionne sans crash.
2. Maîtrise de la Data (Supabase)	/5	Les 3 tables relationnelles sont bien construites. Les règles RLS sont activées (isolation des données entre utilisateurs). L'upload du Storage fonctionne.
3. Déploiement & Git (DevOps)	/3	L'intégration continue Vercel est fonctionnelle. Le dépôt GitHub montre une collaboration réelle (commits réguliers, branches).
4. Qualité UI/UX (Le "Vibe")	/3	L'interface est propre, intuitive et s'adapte correctement au métier choisi par la roulette des thèmes.
5. Rapport Architecte (README)	/4	Le mapping est clair. L'analyse démontre une vraie compréhension des concepts du cours (CAPEX/OPEX, Structuré/Non-structuré, Serverless).